VHDL Assignment 3

Subject: Digital Logic Design

Student ID: 2023024248

Name: Sun Ki Hyun

Contents

1. RS Latch
    1.1 VHDL Source Code
    1.2 Circuit Configuration
    1.3 Simulation Results
2. D Latch
    2.1 VHDL Source Code
    2.2 Circuit Configuration
    2.3 Simulation Results
3. D Flip-Flop
    3.1 VHDL Source Code
    3.2 Circuit Configuration
    3.3 Simulation Results
4. JK Flip-Flop
    4.1 VHDL Source Code
    4.2 Circuit Configuration
    4.3 Simulation Results
5. D Flip-Flop with Asynchronous Set and Clear

5.1 VHDL Source Code
5.2 Simulation Results

6. Conclusion

1. RS Latch

1.1 VHDL Source Code

```vhdl
library ieee;
use ieee.std_logic_1164.all;

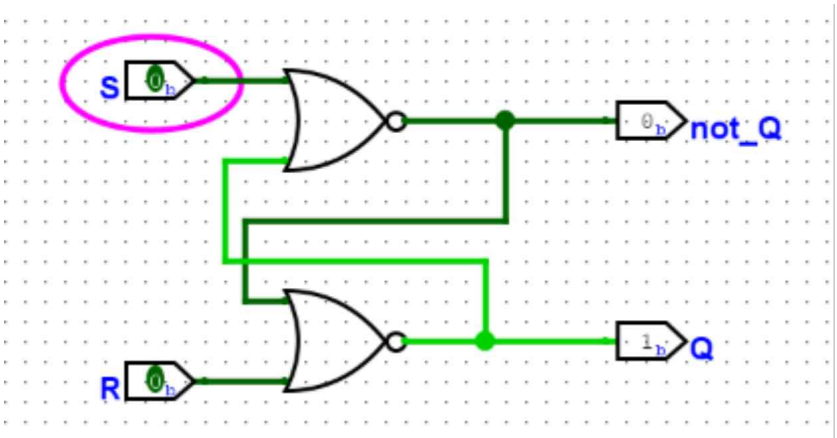
entity rs_latch is
    port (
        S  : in  std_logic;
        R  : in  std_logic;
        Qn : out std_logic;
        Q  : out std_logic
    );
end rs_latch;

architecture gate_level of rs_latch is
    signal q_int  : std_logic;
    signal qn_int : std_logic;
begin
    -- upper NOR gate : S, Q  -> Qn
    qn_int <= S nor q_int;

    -- lower NOR gate : R, Qn -> Q
    q_int  <= R nor qn_int;

    Qn <= qn_int;
    Q  <= q_int;
end gate_level;
```

1.2 Circuit Configuration

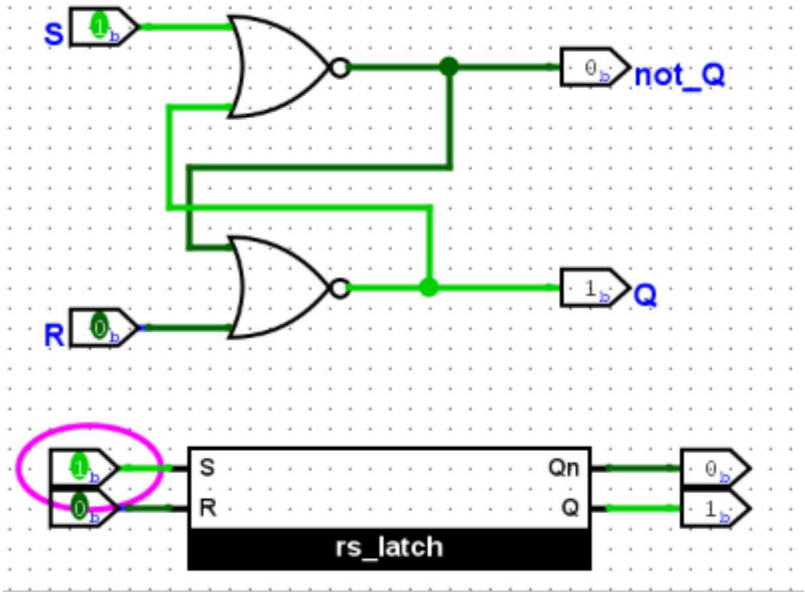


The RS latch is made of two NOR gates connected to each other. The output of each gate goes back to the input of the other gate, so the circuit can store one bit.

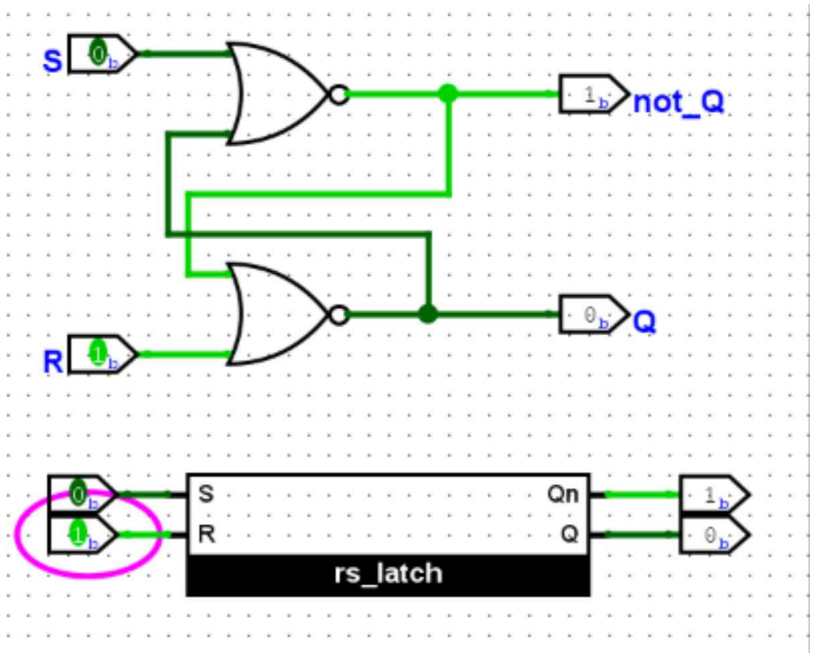
S	R	Q	Qn	Mode
0	0	Q	Qn	Hold
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Forbidden

Truth table of RS latch

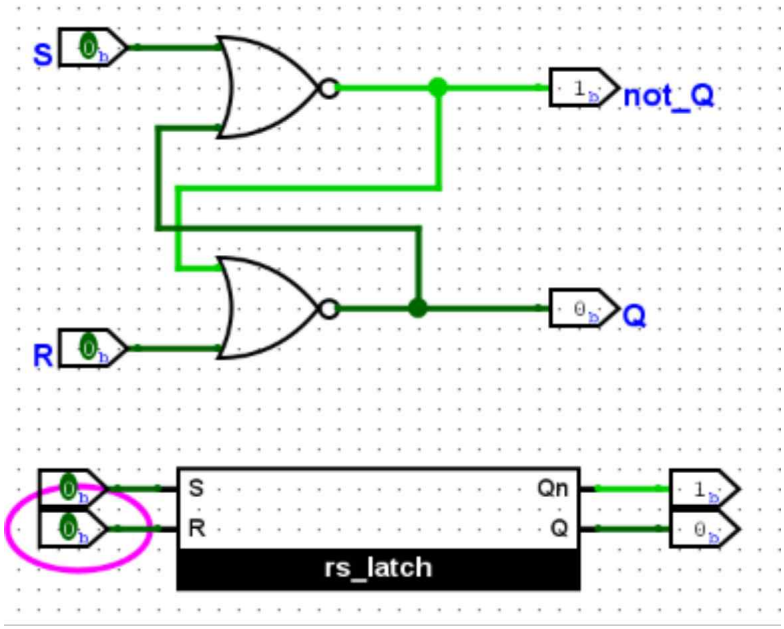
1.3 Simulation Results



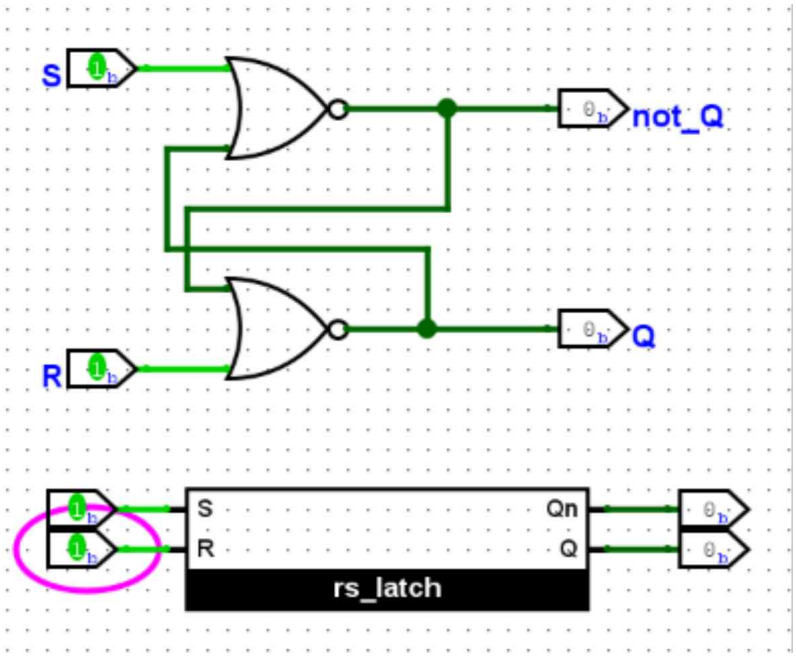
case 1) S = 1, R = 0 : Set, Q = 1, Qn = 0



case 2) S = 0, R = 1 : Reset, Q = 0, Qn = 1



case 3) S = 0, R = 0 : Hold, Q stays 0



case 4) S = 1, R = 1 : Forbidden, Q = 0, Qn = 0

When S is 1, Q becomes 1, and when R is 1, Q becomes 0. When both inputs are 0, Q keeps its previous value. When both inputs are 1, Q and Qn both become 0, so this input should not be used.

2. D Latch

2.1 VHDL Source Code

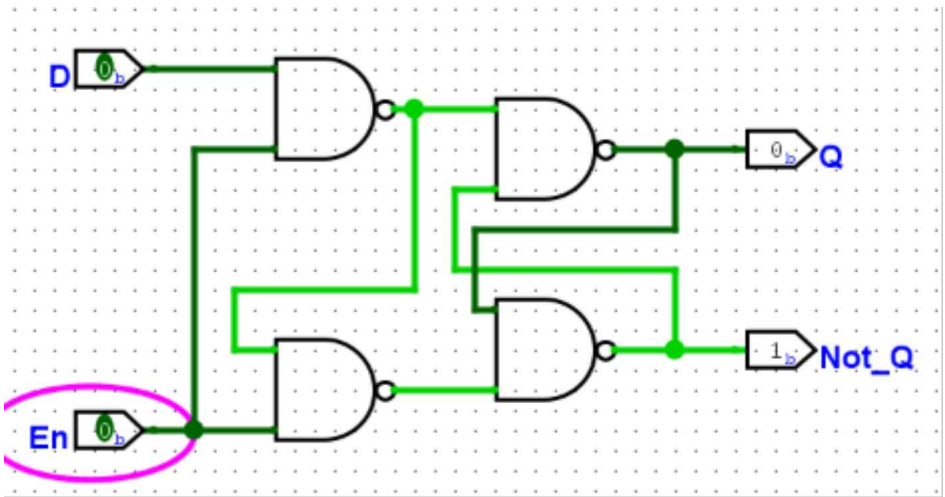
```vhdl
-- ============================================================
-- Exercise 2 : D Latch (with Enable, level-sensitive)
--   en = '1' : Q follows D (transparent)
--   en = '0' : keep previous state (Hold)
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity d_latch is
    port (
        D  : in  std_logic;
        en : in  std_logic;
        Q  : out std_logic;
        Qn : out std_logic
    );
end d_latch;

architecture behavioral of d_latch is
    signal q_int : std_logic := '0';
begin
    process (D, en)
    begin
        if en = '1' then
            q_int <= D;
        end if;           -- en = '0' : no assignment -> Hold
    end process;

    Q  <= q_int;
    Qn <= not q_int;
end behavioral;
```

2.2 Circuit Configuration

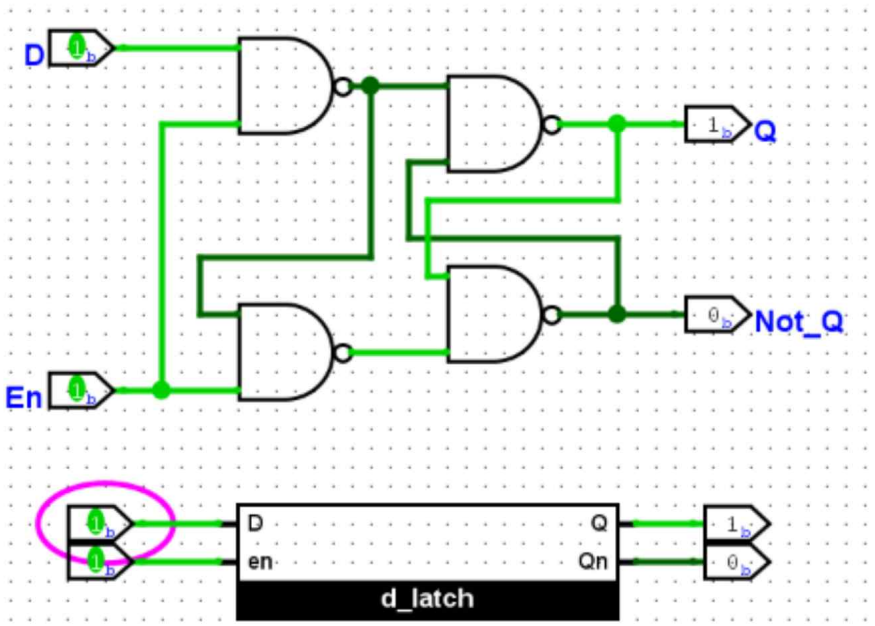


The D latch is made of four NAND gates. The input D passes to the output only when En is 1.

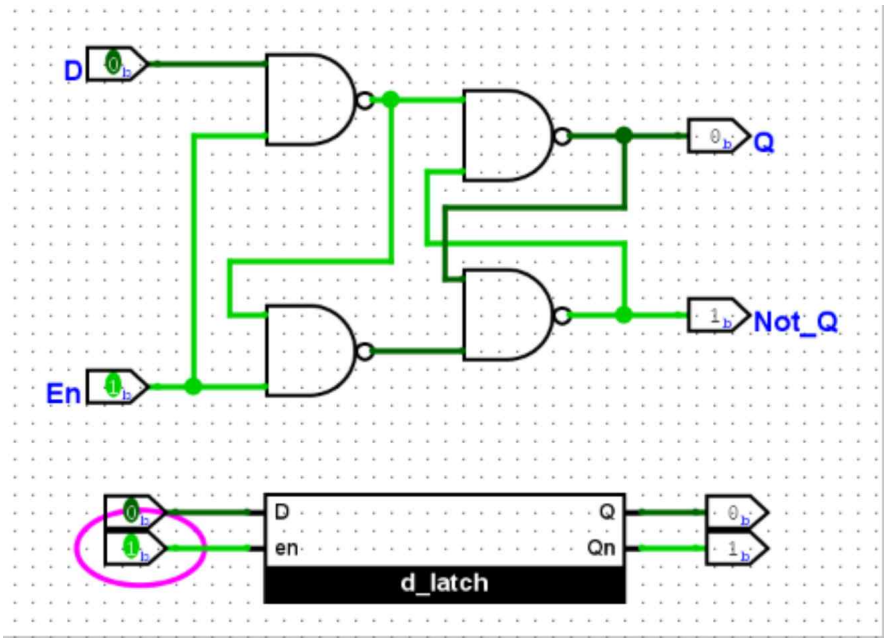
en	D	Q (next)	Mode
0	X	Q	Hold
1	0	0	Reset
1	1	1	Set

Truth table of D latch

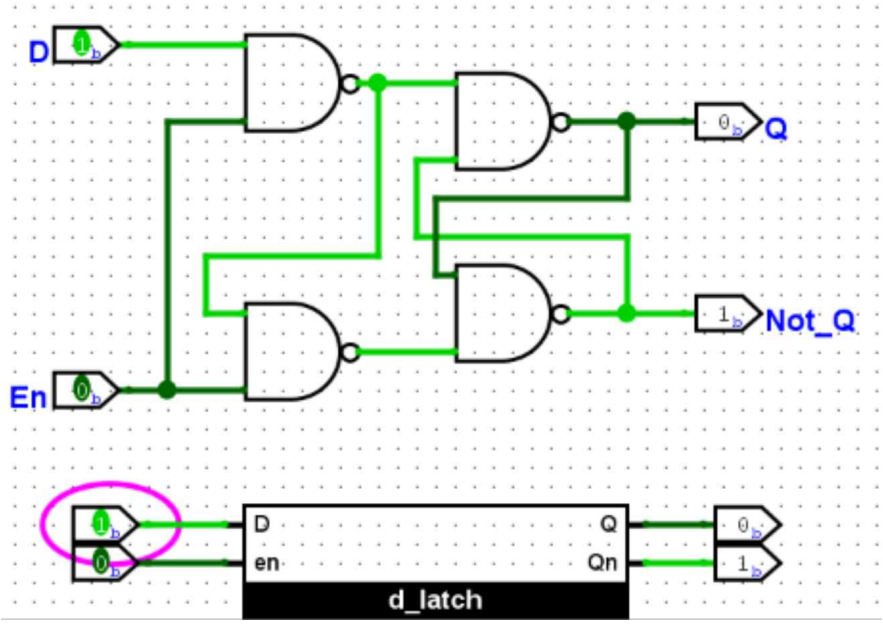
2.3 Simulation Results



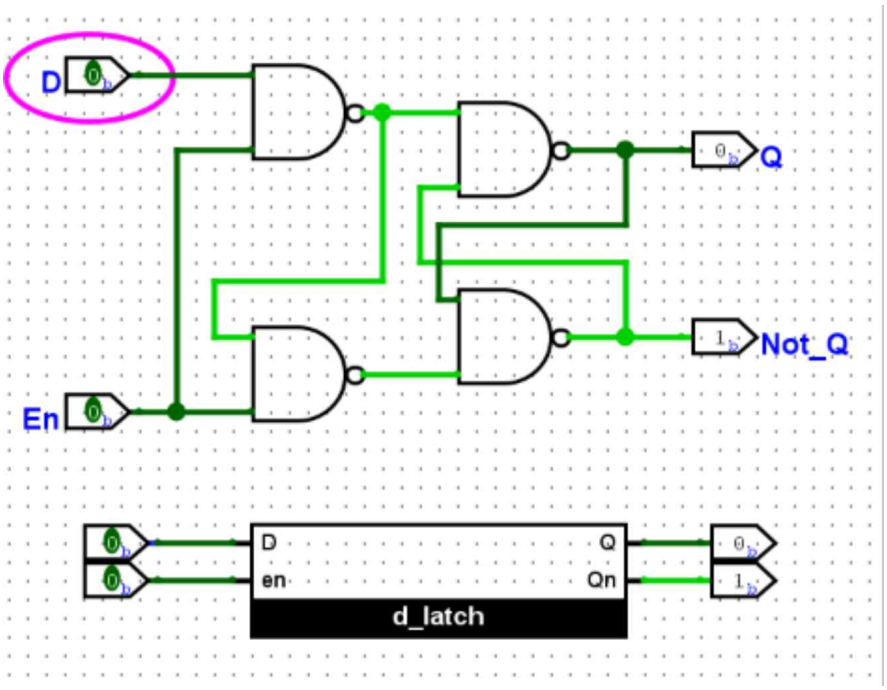
case 1) en = 1, D = 1 : Q = 1



case 2) en = 1, D = 0 : Q = 0



case 3) en = 0, D = 1 : Q stays 0



case 4) en = 0, D = 0 : Q stays 0

When en is 1, Q follows D. When en is 0, Q keeps its previous value even if D changes.

3. D Flip-Flop

3.1 VHDL Source Code

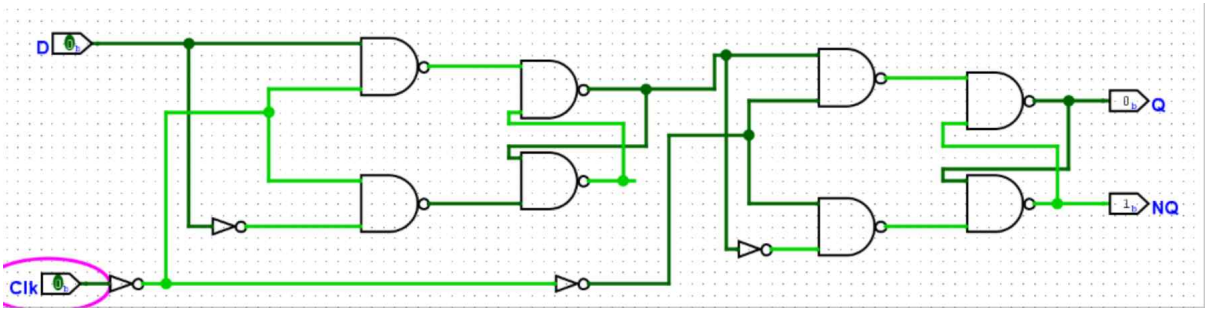
```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity d_ff2 is
    port (
        D   : in  std_logic;
        clk : in  std_logic;
        Q   : out std_logic;
        Qn  : out std_logic
    );
end d_ff2;

architecture behavioral of d_ff2 is
    signal q_int : std_logic := '0';
begin
    process (clk)
    begin
        if rising_edge(clk) then
            q_int <= D;
        end if;
    end process;

    Q  <= q_int;
    Qn <= not q_int;
end behavioral;
```

3.2 Circuit Configuration

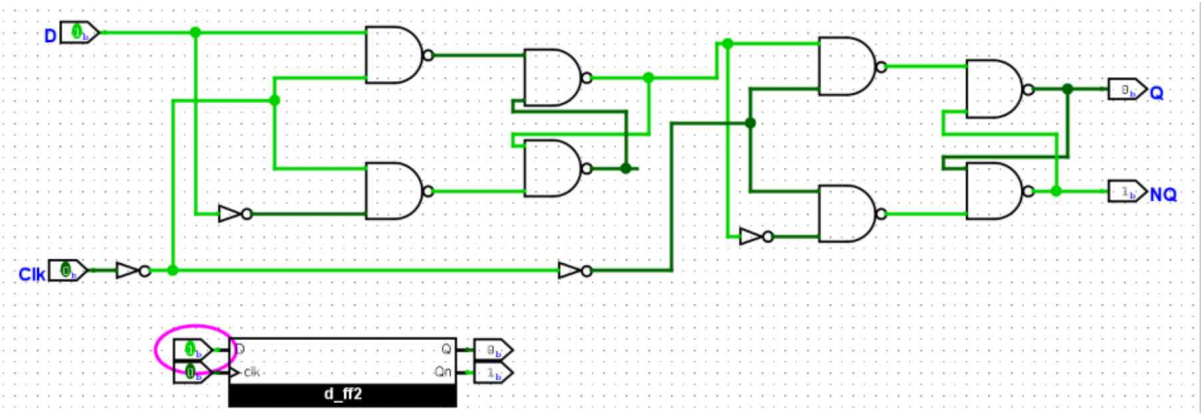


The D flip-flop is made of two D latches and inverters. The first latch works when CLK is 0, and the second latch works when CLK is 1, so Q changes only at the rising edge of CLK.

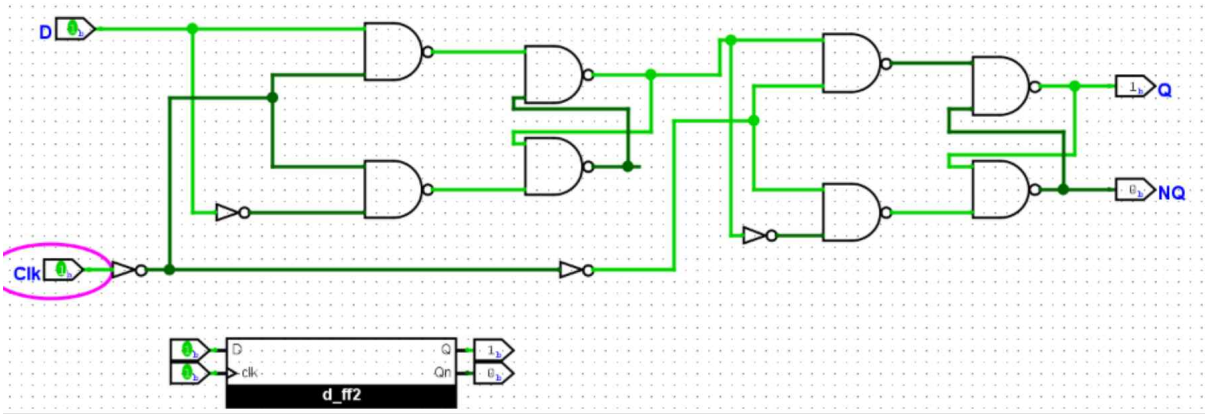
CLK	D	Q (next)
↑	0	0
↑	1	1
0, 1, ↓	X	Q (no change)

Truth table of positive-edge-triggered D flip-flop

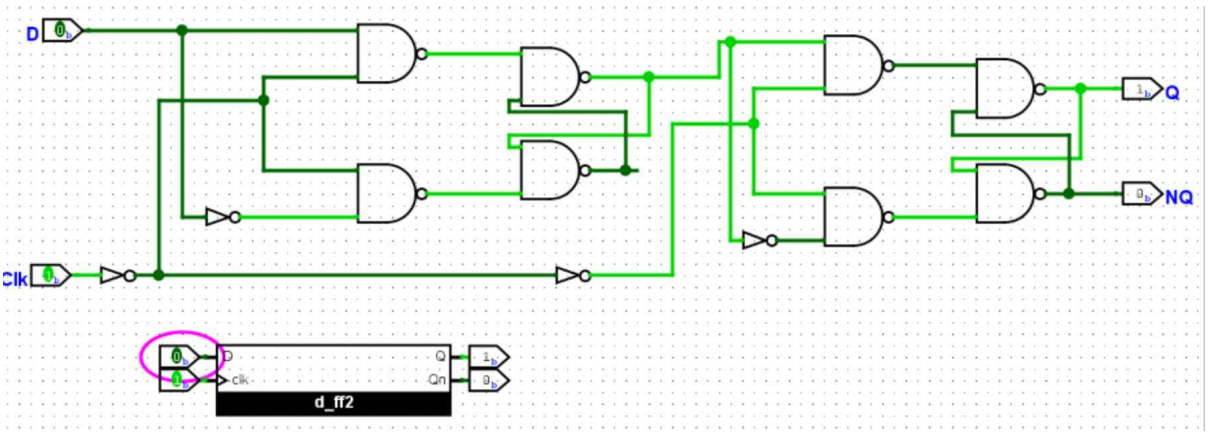
3.3 Simulation Results



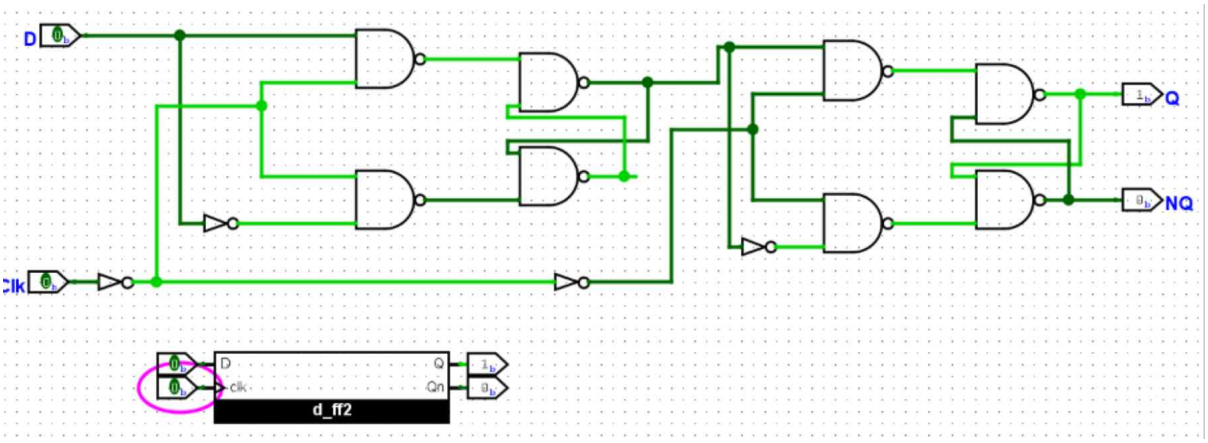
case 1) CLK = 0, D = 1 : Q stays 0



case 2) CLK 0 → 1, D = 1 : Q = 1



case 3) CLK = 1, D = 0 : Q stays 1



case 4) CLK 1 → 0, D = 0 : Q stays 1

Q changes only when CLK goes from 0 to 1. If D changes at any other time, Q does not change. This is different from the D latch, which follows D while en is 1.

4. JK Flip-Flop

4.1 VHDL Source Code

```vhdl
library ieee;
use ieee.std_logic_1164.all;

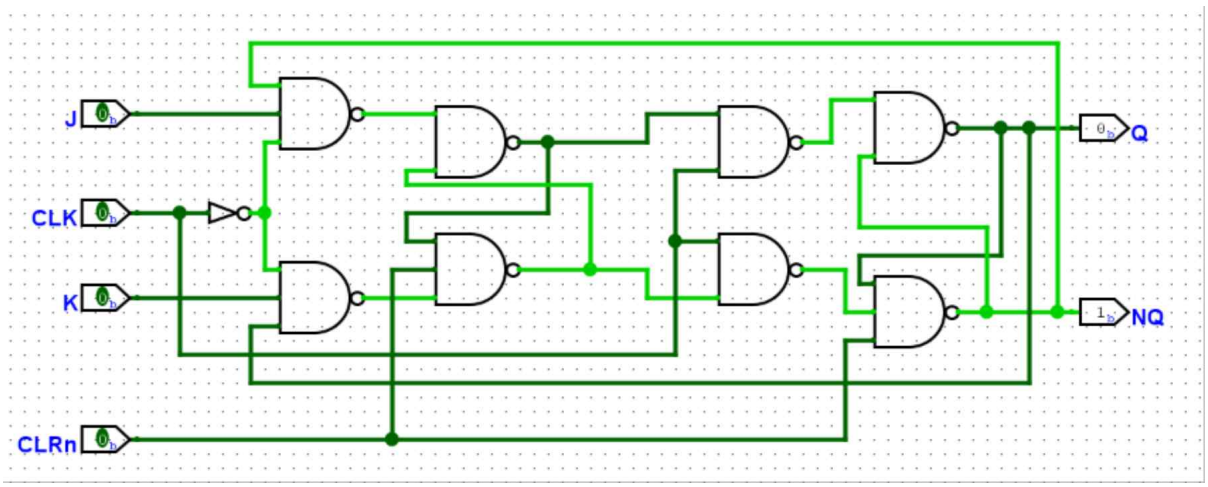
entity jk_ff is
    port (
        J    : in  std_logic;
        clk  : in  std_logic;
        K    : in  std_logic;
        CLRn : in  std_logic;
        Q    : out std_logic;
        Qn   : out std_logic
    );
end jk_ff;

architecture behavioral of jk_ff is
    signal q_int : std_logic := '0';
    signal jk    : std_logic_vector(1 downto 0);
begin
    jk <= J & K;

    -- CLRn is active-low and asynchronous (same as the gate circuit)
    process (clk, CLRn)
    begin
        if CLRn = '0' then
            q_int <= '0';                          -- Clear
        elsif rising_edge(clk) then
            case jk is
                when "00"   => q_int <= q_int;     -- Hold
                when "01"   => q_int <= '0';       -- Reset
                when "10"   => q_int <= '1';       -- Set
                when "11"   => q_int <= not q_int; -- Toggle
                when others => q_int <= q_int;
            end case;
        end if;
    end process;

    Q  <= q_int;
    Qn <= not q_int;
end behavioral;
```

4.2 Circuit Configuration

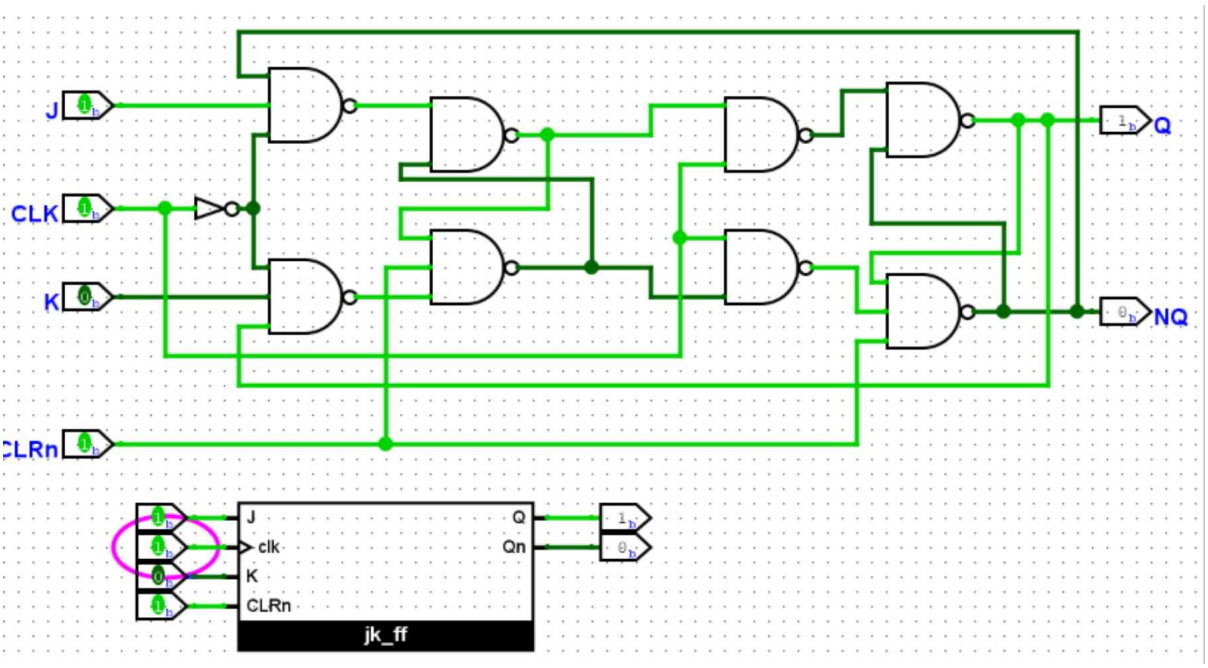


The JK flip-flop is made of eight NAND gates and an inverter in a master-slave structure, so Q changes only at the rising edge of CLK. The CLRn input is used to set Q to 0 at the start of the simulation, and it is kept at 1 during the test.

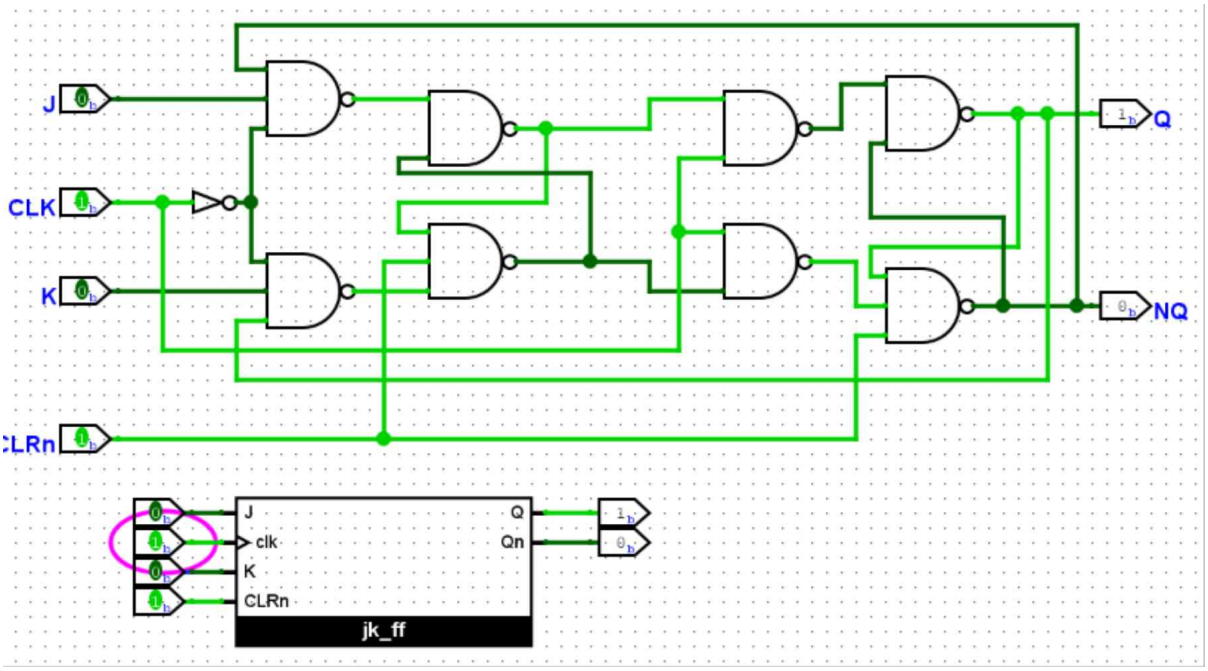
J	K	Q (next)	Mode
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

Truth table of JK flip-flop (at the rising edge of CLK)

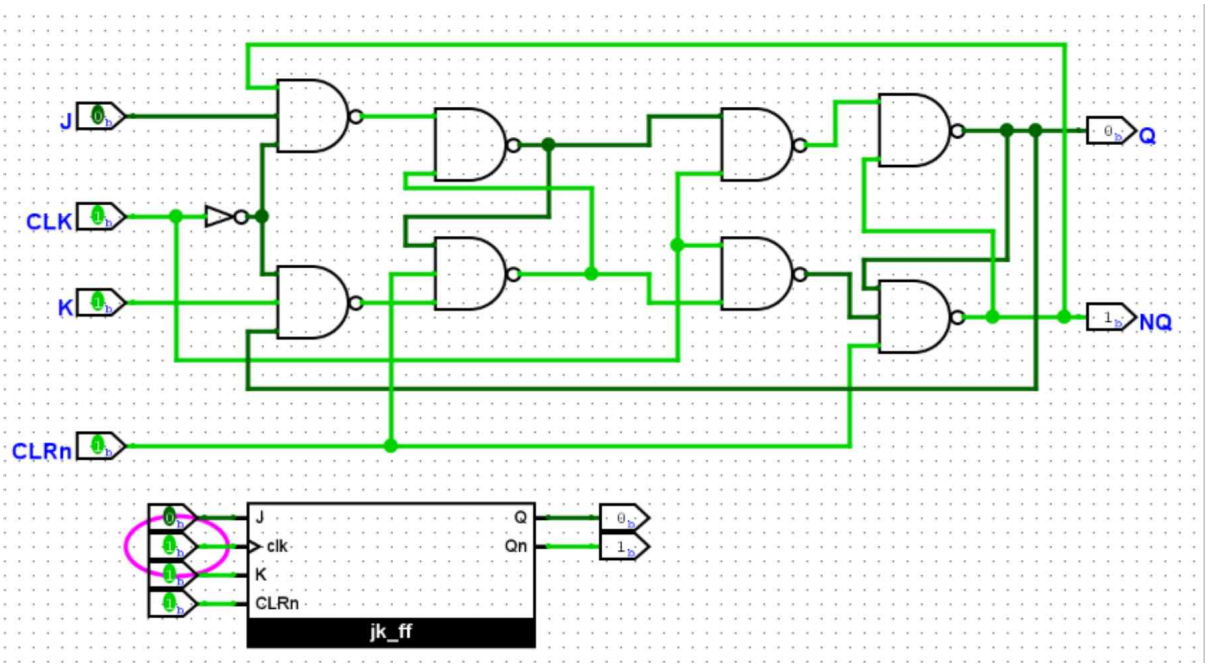
4.3 Simulation Results



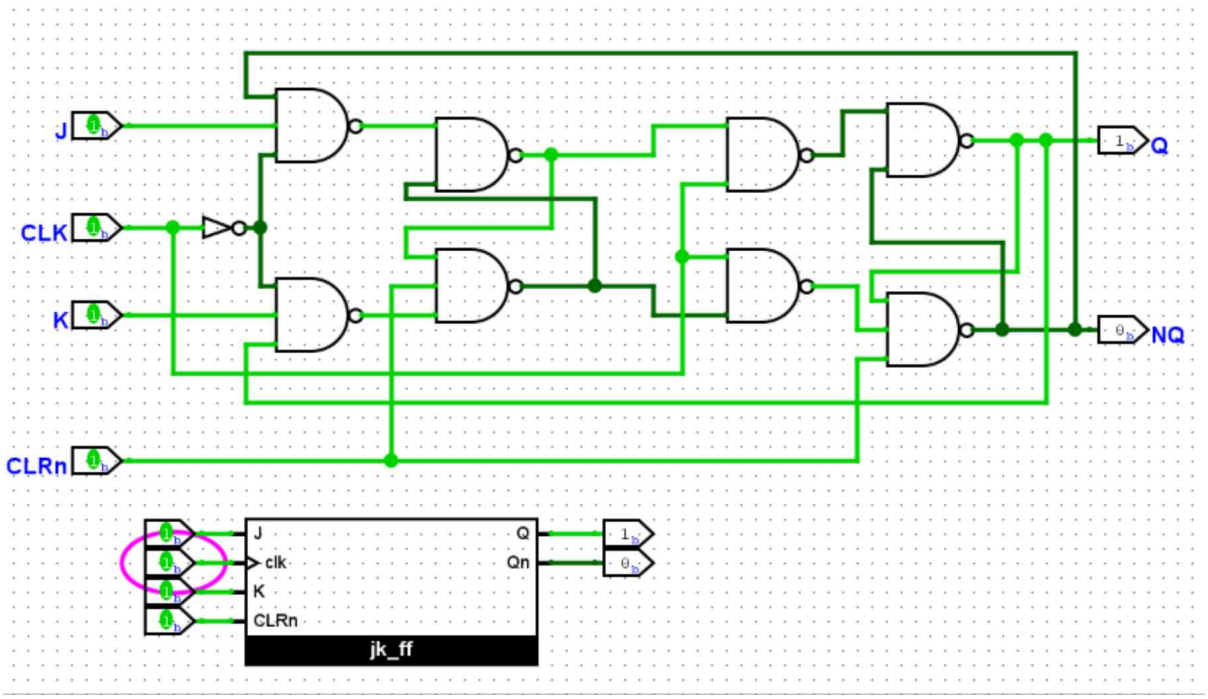
case 1) J = 1, K = 0, CLK ↑ : Set, Q = 1



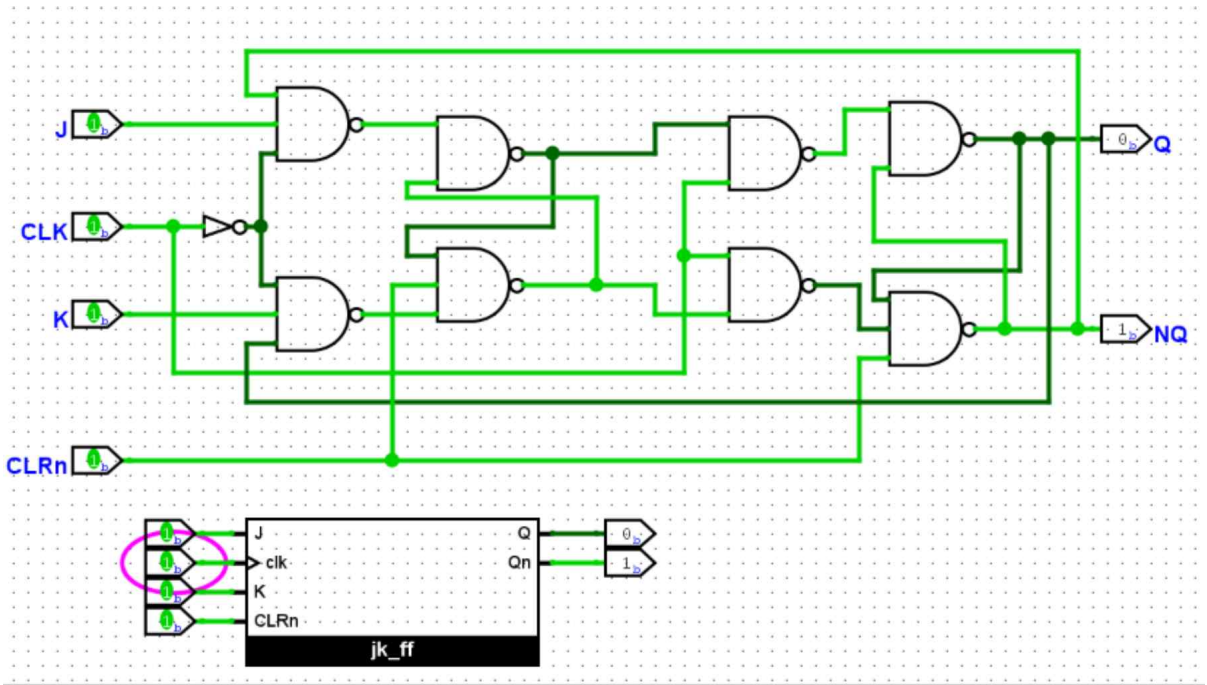
case 2) J = 0, K = 0, CLK ↑ : Hold, Q stays 1



case 3) J = 0, K = 1, CLK ↑ : Reset, Q = 0



case 4) J = 1, K = 1, CLK ↑ : Toggle, Q = 1



case 5) J = 1, K = 1, CLK ↑ : Toggle, Q = 0

At the rising edge of CLK, the circuit holds Q when J = K = 0, resets Q when K = 1, sets Q when J = 1, and toggles Q when J = K = 1.

5. D Flip-Flop with Asynchronous Set and Clear

5.1 VHDL Source Code

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity d_ff_async is
    port (
        clk : in  std_logic;
        D   : in  std_logic;
        set : in  std_logic;
        clr : in  std_logic;
        Q   : out std_logic;
        Qn  : out std_logic
    );
end d_ff_async;

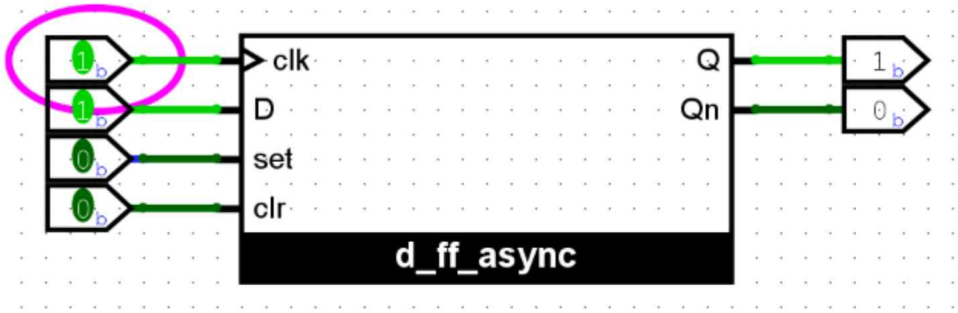
architecture behavioral of d_ff_async is
    signal q_int : std_logic := '0';
begin
    -- set and clr MUST be in the sensitivity list
    -- so the FF reacts immediately without waiting for clk
    process (clk, set, clr)
    begin
        if clr = '1' then              -- 1st priority : Clear
            q_int <= '0';
        elsif set = '1' then           -- 2nd priority : Set
            q_int <= '1';
        elsif rising_edge(clk) then    -- 3rd : normal D FF operation
            q_int <= D;
        end if;
    end process;

    Q  <= q_int;
    Qn <= not q_int;
end behavioral;
```

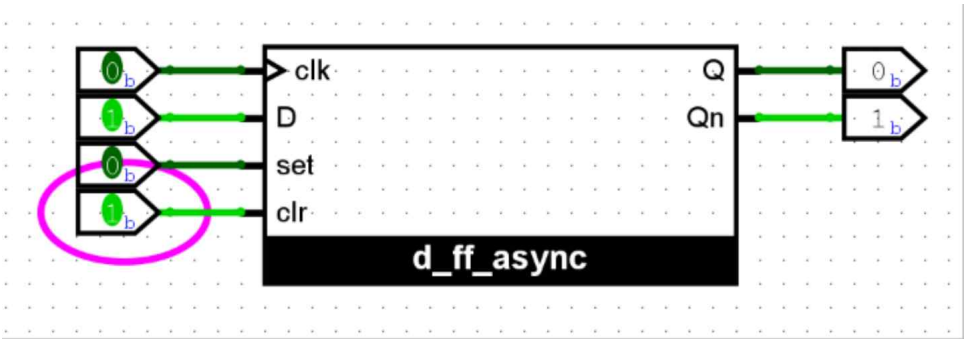
clr	set	CLK	D	Q (next)
1	X	X	X	0
0	1	X	X	1
0	0	↑	0	0
0	0	↑	1	1
0	0	0, 1, ↓	X	Q

Truth table of D flip-flop with asynchronous set and clear

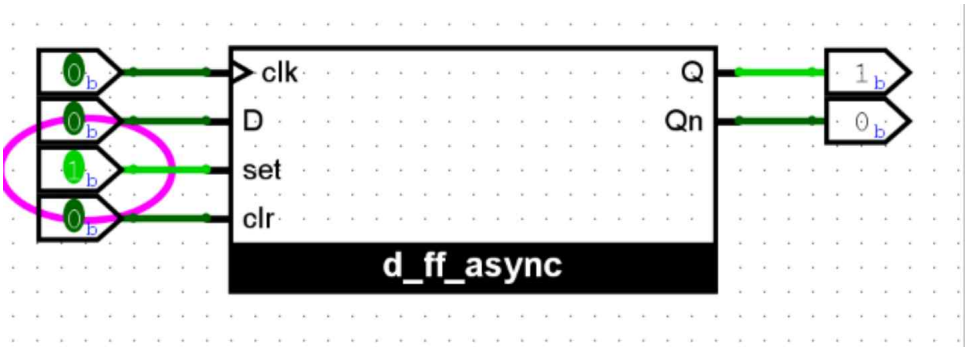
5.2 Simulation Results



case 1) D = 1, CLK ↑ : Q = 1



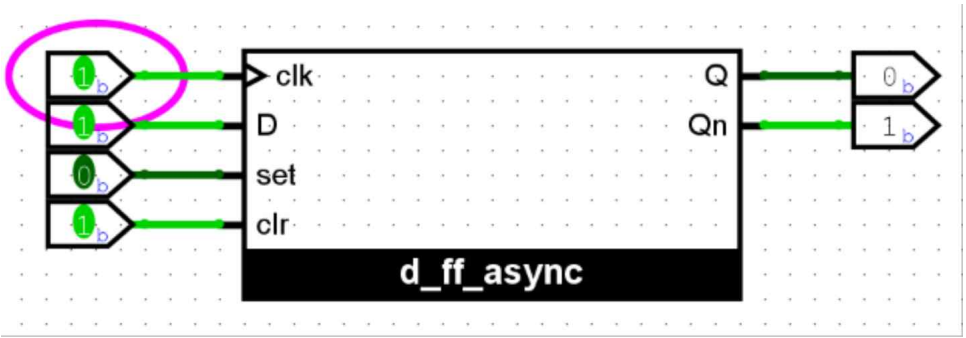
case 2) clr = 1 : Q = 0



case 3) set = 1 : Q = 1



case 4) set = 1, clr = 1 : Q = 0



case 5) clr = 1, D = 1, CLK ↑ : Q stays 0

Q changes right away when clr or set is 1, without waiting for the clock. When clr and set are both 1, Q becomes 0 because clr has higher priority than set.

6. Conclusion

In this assignment, all five circuits were designed in VHDL, and the first four circuits were also built with logic gates in Logisim. The results of the gate circuits and the VHDL circuits were the same for all tested inputs.